`store()` member functions, `exchange(), compare_exchange_weak()`, and `compare_exchange_strong()`. They also support the compound assignment operators where appropriate: `+=, -=, *=, |=,` and so on, and the integral types and `std::atomic<>` specializations for pointers support `++` and `--`. These operators also have corresponding named member functions with the same functionality: `fetch_add(), fetch_or()`, and so on. The return value from the assignment operators and member functions is either the value stored (in the case of the assignment operators) or the value prior to the operation (in the case of the named functions). This avoids the potential problems that could stem from the usual habit of such assignment operators returning a reference to the object being assigned to. In order to get the stored value from such a reference, the code would have to perform a separate read, thus allowing another thread to modify the value between the assignment and the read and opening the door for a race condition.

The `std::atomic<>` class template isn't just a set of specializations, though. It does have a primary template that can be used to create an atomic variant of a user-defined type. Because it's a generic class template, the operations are limited to `load(), store()` (and assignment from and conversion to the user-defined type), `exchange(), compare_exchange_weak()`, and `compare_exchange_strong()`.

Each of the operations on the atomic types has an optional memory-ordering argument that can be used to specify the required memory-ordering semantics. The precise semantics of the memory-ordering options are covered in section 5.3. For now, it suffices to know that the operations are divided into three categories:

- *Store* operations, which can have `memory_order_relaxed, memory_order_release`, or `memory_order_seq_cst` ordering
- *Load* operations, which can have `memory_order_relaxed, memory_order_consume, memory_order_acquire`, or `memory_order_seq_cst` ordering
- *Read-modify-write* operations, which can have `memory_order_relaxed, memory_order_consume, memory_order_acquire, memory_order_release, memory_order_acq_rel`, or `memory_order_seq_cst` ordering

The default ordering for all operations is `memory_order_seq_cst`.

Let's now look at the operations you can actually do on each of the standard atomic types, starting with `std::atomic_flag`.

5.2.2. Operations on std::atomic_flag

`std::atomic_flag` is the simplest standard atomic type, which represents a Boolean flag. Objects of this type can be in one of two states: set or clear. It's deliberately basic and is intended as a building block only. As such, I'd never expect to see it in use, except under very special circumstances. Even so, it will serve as a starting point for discussing the other atomic types, because it shows some of the general policies that apply to the atomic types.